



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 1
ระหว่างวันที่ 26 มีนาคม ถึง 3 เมษายน พ.ศ. 2565

ฟังก์ชัน (Functions)

อ.ลือพล พิพานเมฆาภรณ์

luepol.p@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

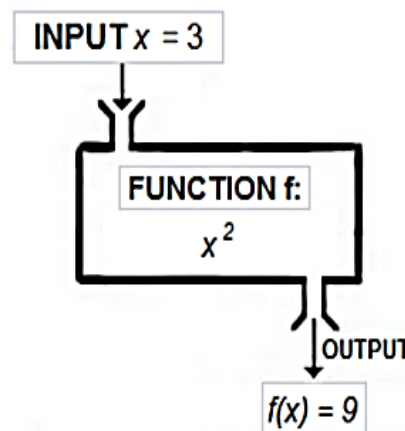
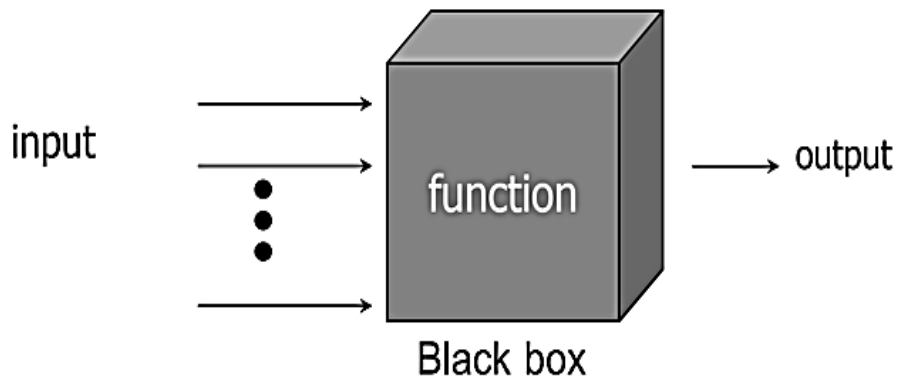
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

การเขียนโปรแกรมแบบแบ่งฟังก์ชัน

- โปรแกรมที่สามารถนำไปงานได้มักจะมีรายละเอียดและมีขนาดใหญ่ ส่งผลให้การปรับปรุงหรือแก้ไขโปรแกรมดังกล่าวทำได้ยากเมื่อโปรแกรมเกิดปัญหา
- ในทางปฏิบัติผู้เขียนโปรแกรมควรต้องแบ่งโปรแกรมออกเป็นโปรแกรมน้อย (subprogram) เพื่อลดความซับซ้อน และยังช่วยให้การตรวจสอบโปรแกรมทำได้ง่ายขึ้น ยิ่งไปกว่านั้นผู้เขียนโปรแกรมยังสามารถแยกพัฒนาโปรแกรมน้อยเหล่านี้รวมทั้งยังสามารถนำไปใช้กับโปรแกรมอื่น (reusable) ได้ด้วย
- ภาษาซีอนุญาตให้ผู้เขียนโปรแกรมสามารถพัฒนาโปรแกรมน้อยเหล่านี้ได้ เรียกว่าฟังก์ชัน (function) หรือ C function

ฟังก์ชันในโปรแกรมภาษาซี

- มีลักษณะคล้ายกับฟังก์ชัน main()
- ฟังก์ชันจะบรรจุชุดคำสั่งสำหรับการทำงานเฉพาะเจาะจง (specific task)
- สามารถรับและส่งข้อมูลจากผู้เรียกใช้งานฟังก์ชันได้



```
int f(int x)
{
    int y;
    y = pow(x, 2);
    return y;
}
```

การทำงานของฟังก์ชัน

Main program

```
int main(void)
{
    func1 ( )
    func2 ( )
    func3 ( )
}
```

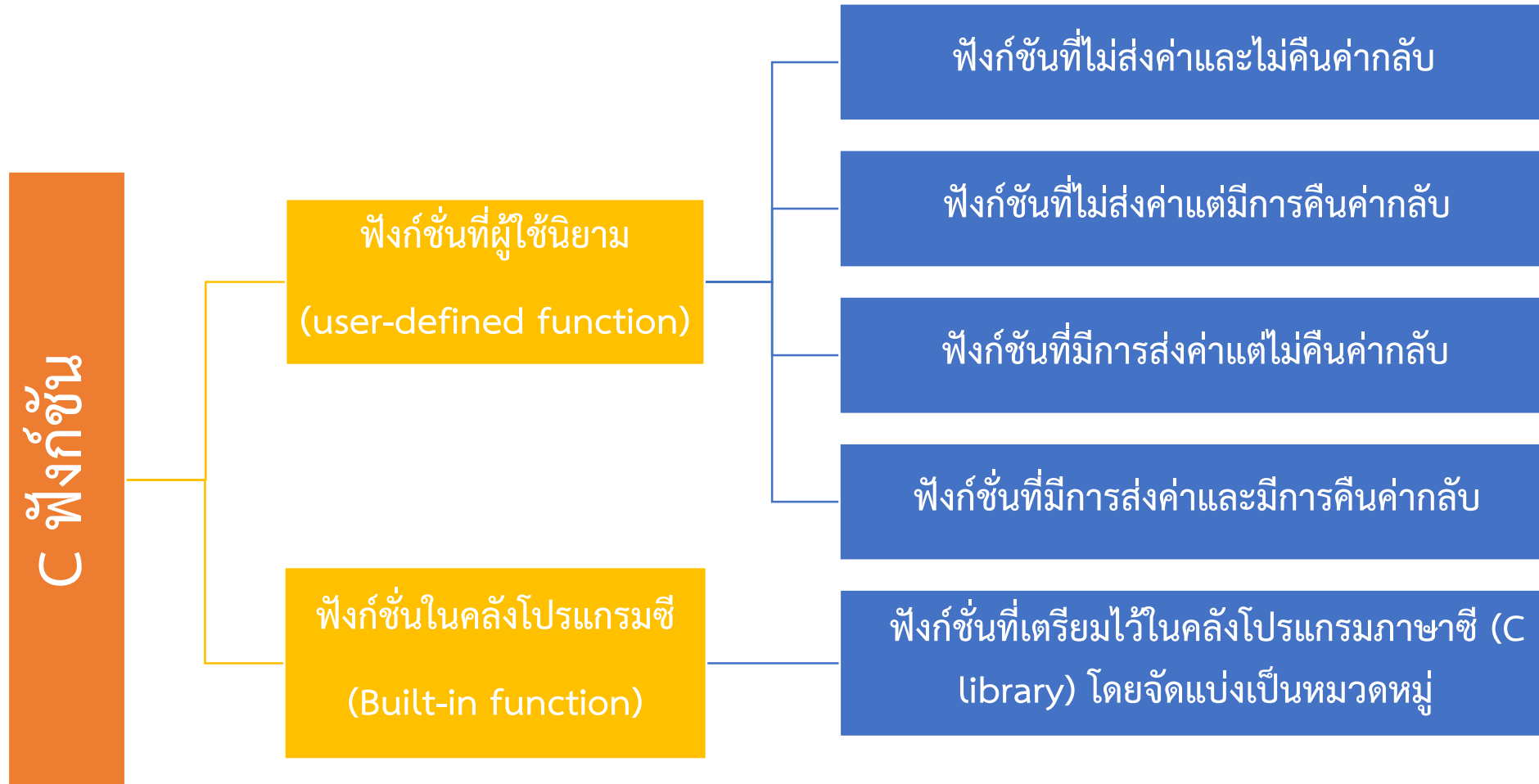
```
func1 ( )
{
    ...
}
```

```
func2 ( )
{
    ...
}
```

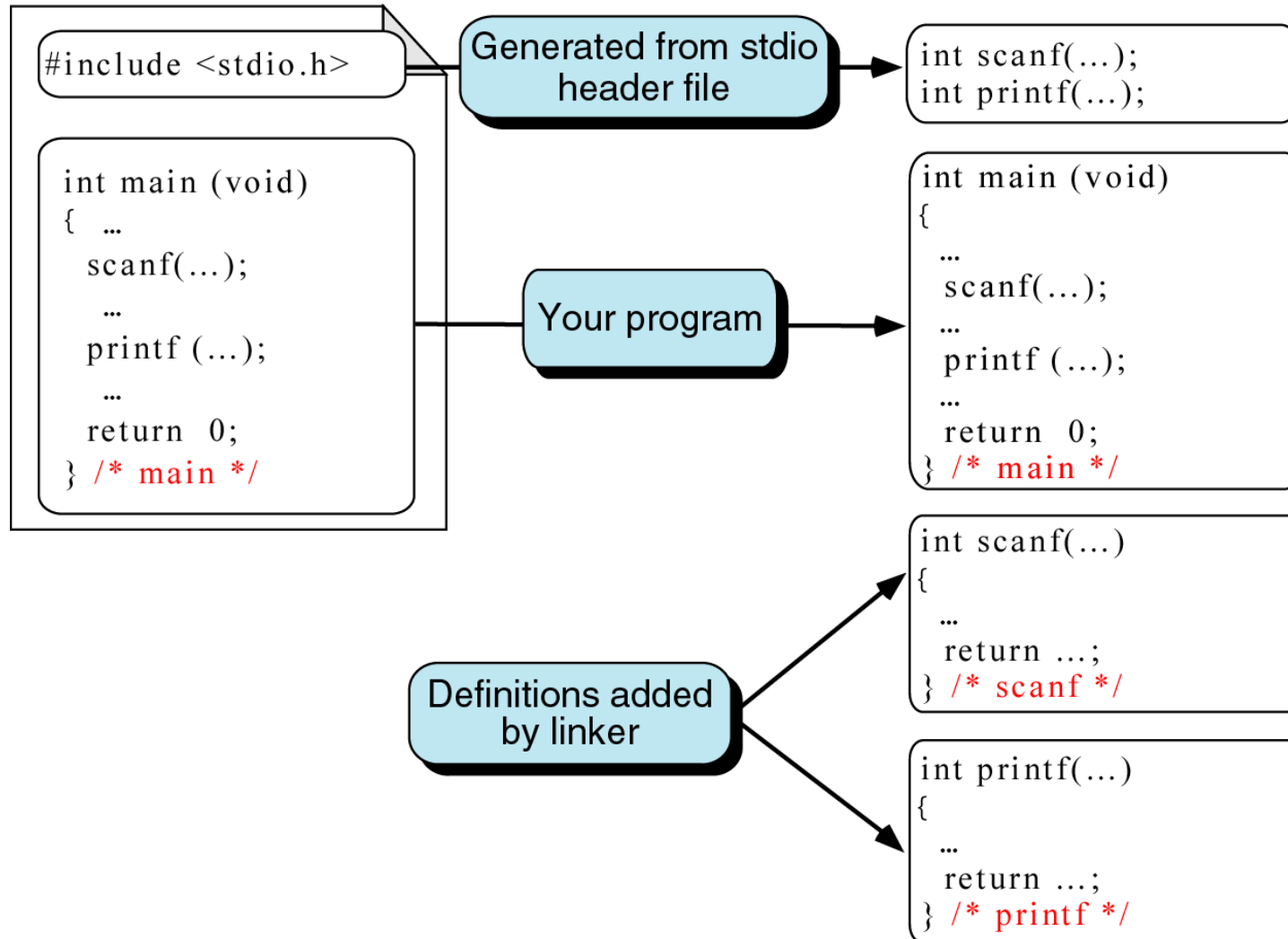
```
func3 ( )
{
    .....
}
```

- ฟังก์ชัน main() จะทำหน้าที่เป็นตัวหลักในการเรียกใช้งานฟังก์ชันอื่นในโปรแกรม
- เมื่อฟังก์ชัน main() เรียกใช้งานฟังก์ชัน โปรแกรมก็จะหยุดการทำงานในฟังก์ชัน main() พร้อมกระโดดไปทำงานในฟังก์ชันที่ถูกเรียกใช้
- เมื่อโปรแกรมทำงานในฟังก์ชันที่ถูกเรียกจนเสร็จ โปรแกรมก็จะกลับมาทำงานต่อใน main() ณ ตำแหน่งที่เรียกใช้ฟังก์ชัน

ประเภทของฟังก์ชันในภาษาซี



C library



- ภาษาซีเตรียมฟังก์ชันมากมายไว้ในคลังโปรแกรมภาษาซี (C library) โดยจัดแบ่งเป็นหมวดหมู่เก็บไว้ในรูปแบบไฟล์ header (.h)
- เช่น `scanf(...)`; ทำหน้าที่รับค่าข้อมูลจากผู้ใช้งานผ่านคีย์บอร์ด
- `printf(..)`; ทำหน้าที่แสดงผลค่าตัวแปรหรือแสดง text ออกทางหน้าจอภาพ

ตัวอย่างของไลบรารีภาษาซี

- ctype.h: character manipulation
- math.h: mathematical functions
- stdio.h: standard input/output
- stdlib.h: random numbers, memory handling
- string.h: string manipulation
- time.h: date/time functions

ตัวอย่างของฟังก์ชันใน math.h

ตัวอย่างฟังก์ชัน	ความหมาย	ชนิดของตัวแปรเข้า	ชนิดของผลลัพธ์
abs(x)	ค่าสัมบูรณ์ของจำนวนเต็ม x	int	int
labs(x)	ค่าสัมบูรณ์ของจำนวนเต็มความจุสูง x	long	long
fabs(x)	ค่าสัมบูรณ์ของจำนวนจริง x	double	double
atan(x)	ค่า arctangent ของ x	double	double
sin(x)	ค่า sine ของ x	double	double
cos(x)	ค่า cosine ของ x	double	double
pow(x, y)	ค่า x ยกกำลัง y	double	double
rand()	สุ่มเลขจำนวนเต็มตั้งแต่ 0 ถึง 32,767	-	int



ตัวอย่างการเรียกใช้ฟังก์ชัน

```
#include <stdio.h>
#include <math.h>
```

```
int main () {
    int i, n;
```

```
    n = 5;
```

```
    printf("print 5 random numbers from 0 to 49\n");
```

```
    for( i = 0 ; i < n ; i++ ) {
        printf("%d\n", rand() % 50);
    }
```

```
    return 0;
```

```
}
```

A screenshot of a Windows command prompt window titled "C:\Users\ZenBook\Desktop\Untitled1.exe". The window has a black background with white text. The first line of text is "print 5 random numbers from 0 to 49". This is followed by five lines of random numbers: 41, 17, 34, 0, and 19. Below the numbers, the text "Process returned 0 (0x0) execution time : 0.028 s" is displayed, followed by "Press any key to continue." on the last line. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

```
C:\Users\ZenBook\Desktop\Untitled1.exe
print 5 random numbers from 0 to 49
41
17
34
0
19
Process returned 0 (0x0) execution time : 0.028 s
Press any key to continue.
```

ตัวอย่างของฟังก์ชันใน string.h

ตัวอย่างฟังก์ชัน	ความหมาย	ชนิดของตัวแปรเข้า	ชนิดของผลลัพธ์
strlen(s)	ความยาวสตริง s	string	int
strcpy(str2, str1)	คัดลอกสตริงจาก str1 ไปยัง str2	string	-
strcat(a, b)	เชื่อมสตริง a โดยนำสตริง b ไปต่อท้าย	string	-
strcmp(a, b)	เปรียบเทียบสตริง a และ b ว่ามีความเหมือนกันหรือแตกต่างกัน	string	int

ตัวอย่างการเรียกใช้ฟังก์ชัน

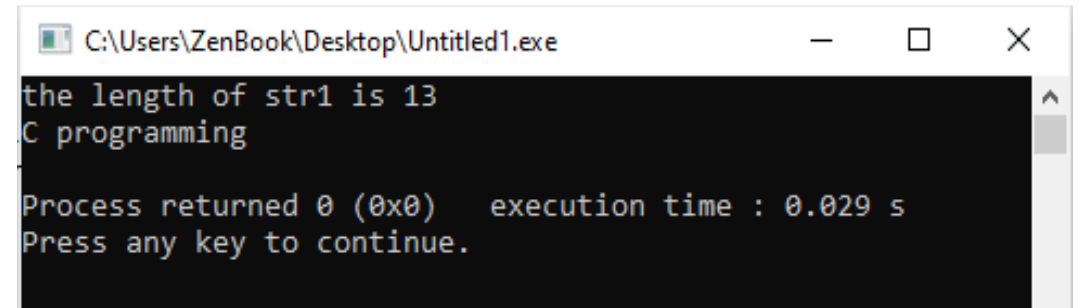
```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char str1[20] = "C programming";
    char str2[20];
```

```
    printf("the length of str1 is %d\n", strlen(str1));
```

```
    strcpy(str2, str1);
    puts(str2);
```

```
    return 0;
}
```

A screenshot of a Windows command prompt window titled 'C:\Users\ZenBook\Desktop\Untitled1.exe'. The window shows the output of the C program: 'the length of str1 is 13' and 'C programming'. Below the output, it says 'Process returned 0 (0x0) execution time : 0.029 s' and 'Press any key to continue.'.

```
C:\Users\ZenBook\Desktop\Untitled1.exe
the length of str1 is 13
C programming

Process returned 0 (0x0)   execution time : 0.029 s
Press any key to continue.
```

ฟังก์ชันที่ผู้ใช้นิยาม (user-defined function)

- นอกจากฟังก์ชันในคลังโปรแกรมภาษาซี ภาษาซีก็ยังอนุญาตให้ผู้เขียนโปรแกรมสามารถพัฒนาฟังก์ชันขึ้นมาเพื่อใช้งานในโปรแกรมได้ด้วย โดยมีการประกาศและนิยามฟังก์ชัน
- องค์ประกอบของฟังก์ชันที่ผู้ใช้นิยามในภาษาซี
 1. การประกาศฟังก์ชัน (function prototype)
 2. การนิยามฟังก์ชัน (function definition)
 3. การเรียกใช้งานฟังก์ชัน (function call)



- return type function_name (param 1, ..., param n);
- ↑
- ชื่อฟังก์ชัน(function name) รายการพารามิเตอร์ (parameter list)
- ชนิดของข้อมูลที่จะส่งกลับ (function return type)

การประกาศฟังก์ชัน

■ ชื่อฟังก์ชัน (function name)

เป็นชื่อที่ใช้ในการเรียกใช้ฟังก์ชัน ผู้เขียนโปรแกรมควรตั้งชื่อให้ตรงตามวัตถุประสงค์ของฟังก์ชัน

■ รายการพารามิเตอร์ (parameter list)

- รายการชนิดของข้อมูลซึ่งผู้เรียก (caller) จะส่งมาเมื่อมีการเรียกใช้ฟังก์ชัน โดยรายการพารามิเตอร์อาจมีได้ตั้งแต่ 1 ตัวขึ้นไป ทั้งนี้ขึ้นอยู่กับผู้เขียนโปรแกรมออกแบบ
- พารามิเตอร์แต่ละตัวอาจมีการประกาศตัวแปรใช้งานร่วมด้วยได้

■ ชนิดของข้อมูลที่ส่งกลับ (return type)

ระบุชนิดของข้อมูลที่ให้ฟังก์ชันส่งค่ากลับไปยังผู้เรียก เช่น int, float, char, double เป็นต้น หากฟังก์ชันไม่มีการส่งค่าคืนกลับก็จะต้องมีการระบุส่วนของ return type ว่าเป็น **void**

การนิยามฟังก์ชัน (function definition)

- จะเป็นส่วนที่ใช้พัฒนาฟังก์ชันใช้งาน โดยจะต้องสอดคล้องกับการประกาศฟังก์ชัน เช่น จำนวนและชนิดของข้อมูลสำหรับพารามิเตอร์แต่ละตัว รวมทั้งชนิดของข้อมูลที่ใช้ส่งค่ากลับ
- คำสั่ง **return value** จะส่งค่ากลับไปยังผู้เรียกใช้ฟังก์ชัน โดยการระบุเป็น constant, variable หรือ expression ก็ได้ แต่ต้องสอดคล้องกับ return type ที่ประกาศในฟังก์ชัน prototype

```
return type function_name (param 1, ..., param n)
{
    .....
    return value;
}
```

ตัวอย่างการพัฒนาฟังก์ชันในภาษาซี

```
#include <stdio.h>
```

```
int pow_n(int , int );    // function prototype
```

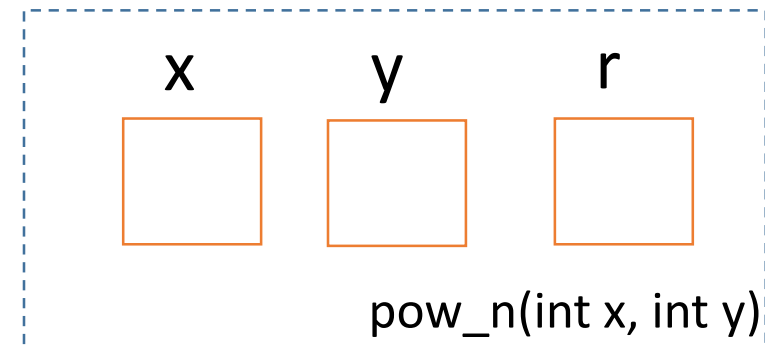
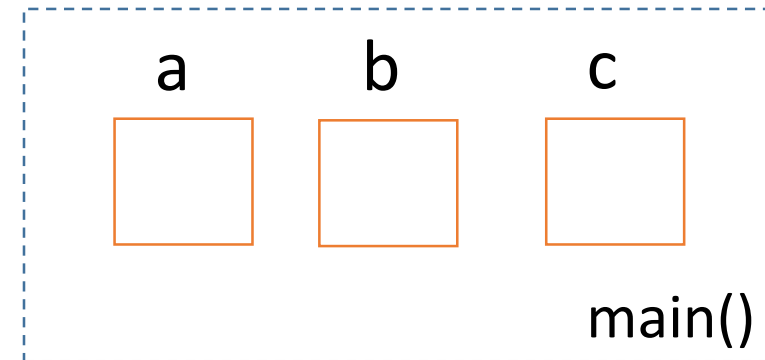
```
int main() {
    int a, b, c;

    scanf("%d %d", &a, &b);
    c = pow_n(a, b);    // function call
    printf("%d ^ %d = %d ", a, b, c);

    return 0;
}
```

```
int pow_n(int x, int y)    // function definition
{ int r=1;

    while(y > 0)
    { r= r * x;
      y--;
    }
    return r;
}
```



ตัวอย่างการพัฒนาฟังก์ชันในภาษาซี

```
#include <stdio.h>
```

```
int pow_n(int , int );    // function prototype
```

```
int main() {
    int a, b, c;
```

```
    scanf("%d %d", &a, &b);
```

```
    c = pow_n(a, b);    // function call
```

```
    printf("%d ^ %d = %d ", a, b, c);
```

```
    return 0;
```

```
}
```

```
int pow_n(int x, int y)    // function definition
{ int r=1;
```

```
    while(y > 0)
```

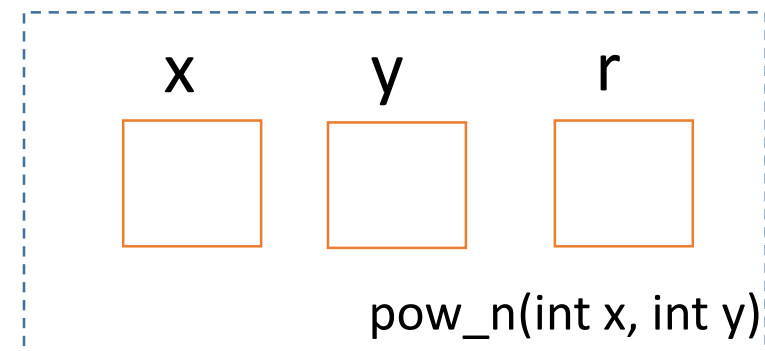
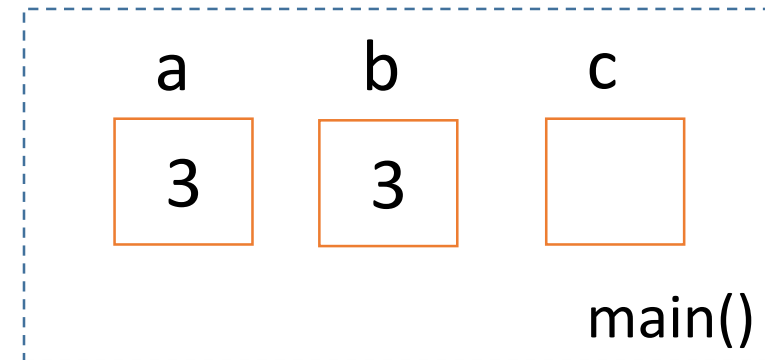
```
    { r= r * x;
```

```
      y--;
```

```
    }
```

```
    return r;
```

```
}
```



ตัวอย่างการพัฒนาฟังก์ชันในภาษาซี

```
#include <stdio.h>

int pow_n(int , int );    // function prototype

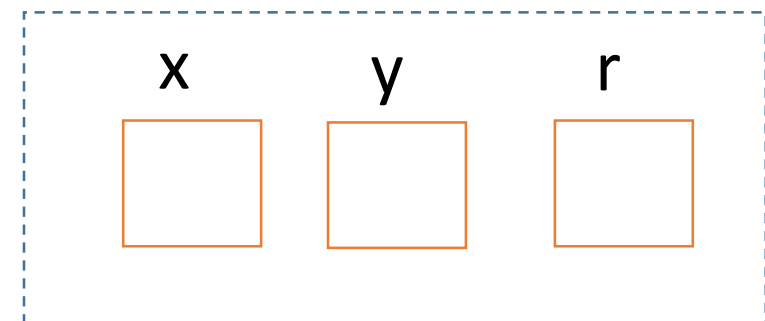
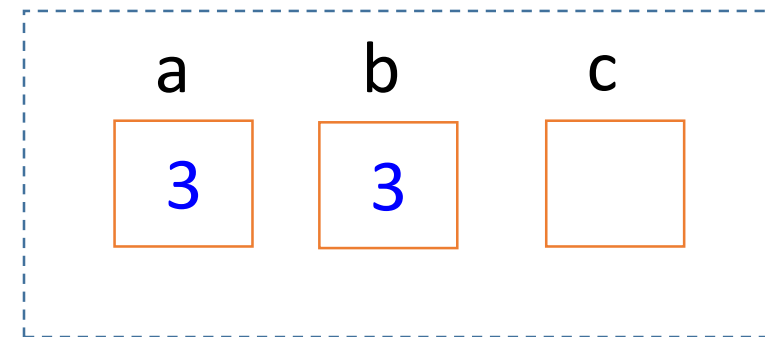
int main() {
    int a, b, c;

    scanf("%d %d", &a, &b);
    c = pow_n(a, b);      // function call
    printf("%d ^ %d = %d ", a, b, c);

    return 0;
}

int pow_n(int x, int y)    // function definition
{ int r=1;

    while(y > 0)
    { r= r * x;
      y--;
    }
    return r;
}
```



ตัวอย่างการพัฒนาฟังก์ชันในภาษาซี

```
#include <stdio.h>

int pow_n(int , int );    // function prototype

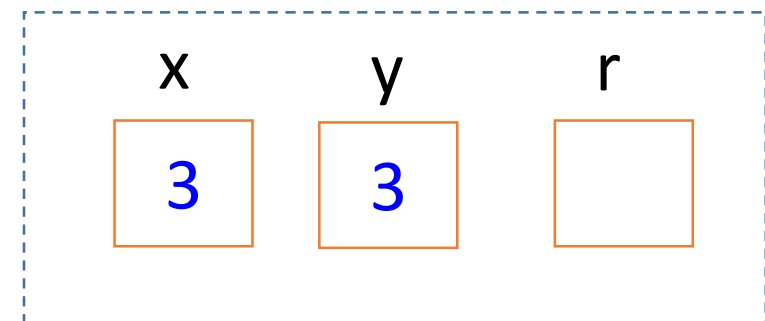
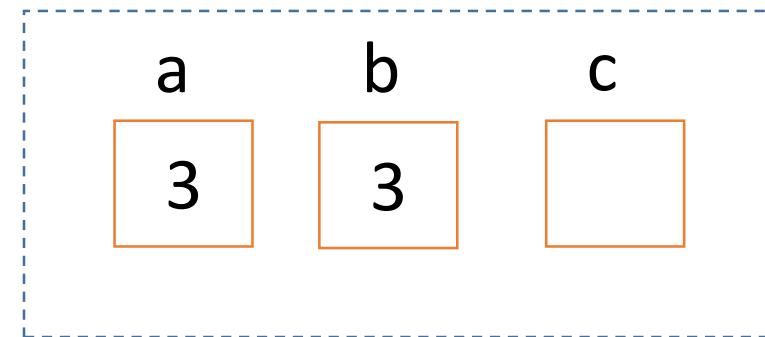
int main() {
    int a, b, c;

    scanf("%d %d", &a, &b);
    c = pow_n(a, b);        // function call
    printf("%d ^ %d = %d ", a, b, c);

    return 0;
}

int pow_n(int x, int y)    // function definition
{ int r=1;

    while(y > 0)
    { r= r * x;
      y--;
    }
    return r;
}
```



ตัวอย่างการพัฒนาฟังก์ชันในภาษาซี

```
#include <stdio.h>

int pow_n(int , int );    // function prototype

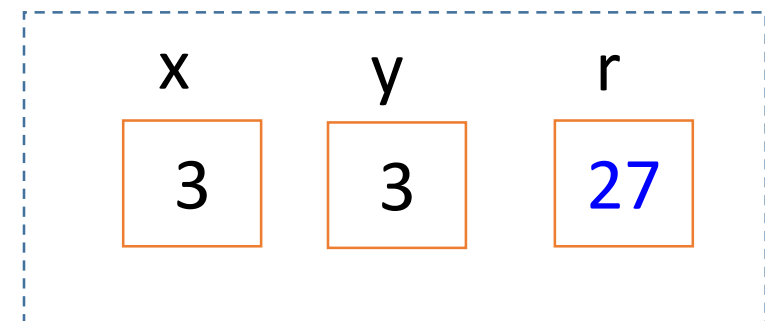
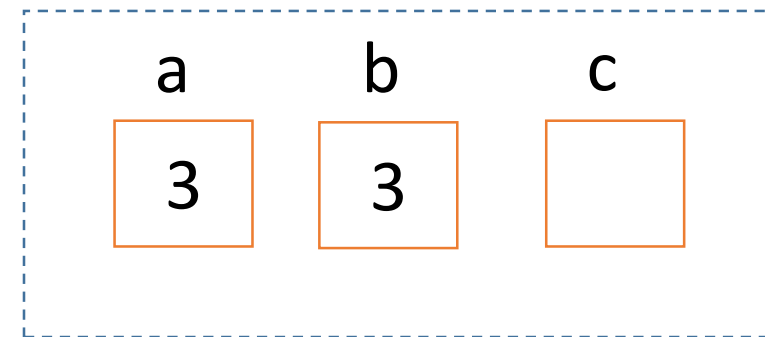
int main() {
    int a, b, c;

    scanf("%d %d", &a, &b);
    c = pow_n(a, b);        // function call
    printf("%d ^ %d = %d ", a, b, c);

    return 0;
}

int pow_n(int x, int y)    // function definition
{ int r=1;

    while(y > 0)
    { r= r * x;
      y--;
    }
    return r;
}
```



ตัวอย่างการพัฒนาฟังก์ชันในภาษาซี

```
#include <stdio.h>

int pow_n(int , int );    // function prototype

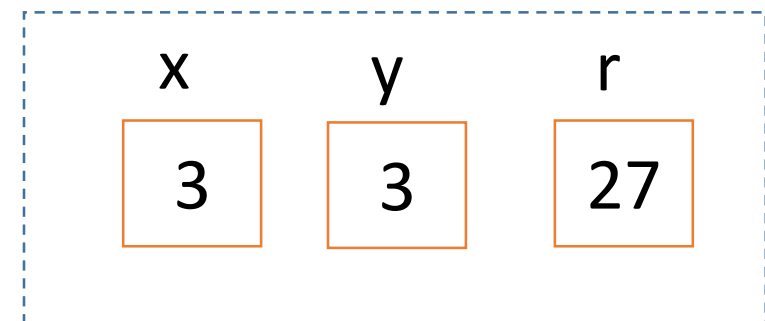
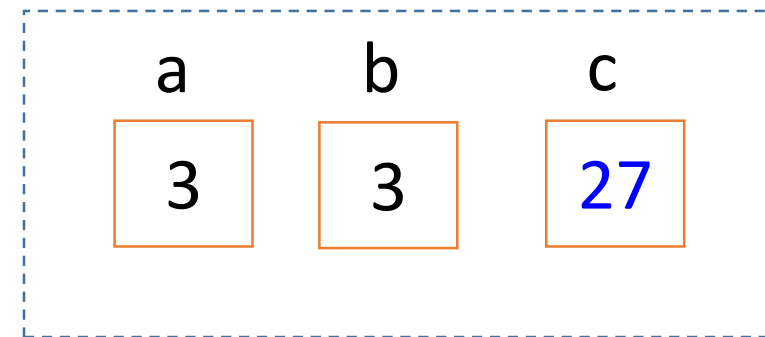
int main() {
    int a, b, c;

    scanf("%d %d", &a, &b);
    c = pow_n(a, b);      // function call
    printf("%d ^ %d = %d ", a, b, c);

    return 0;
}

int pow_n(int x, int y)  // function definition
{ int r=1;

    while(y > 0)
    { r= r * x;
      y--;
    }
    return r;
}
```



รูปแบบของฟังก์ชันที่ผู้ใช้นิยามในภาษาซี

- ฟังก์ชันมีการส่งค่าและคืนค่ากลับ* (have argument and have return)

```
int func(int , int, ....);
```

- ฟังก์ชันมีการส่งค่าแต่ไม่คืนค่ากลับ (have argument but no return)

```
void func(int, int, ....);
```

- ฟังก์ชันไม่ส่งค่าและไม่คืนค่ากลับ (have no argument and no return)

```
void func(void); หรือ void func();
```

- ฟังก์ชันไม่ส่งค่าแต่มีการคืนค่ากลับ (have no argument but return)

```
int func(void);
```

* ฟังก์ชันในภาษาซีอนุญาตให้ส่งค่ากลับได้เพียงค่าเดียวเท่านั้น²²

ตัวอย่างฟังก์ชันที่มีการส่งค่าและมีการคืนค่ากลับ

```
#include <stdio.h>

int max(int a, int b);

int main()
{ int a=2, b=5, c;

  c = max(a, b);
  printf("max is %d \n", c);

  return 0;
}

int max(int a, int b)
{ return (a>b)? a : b;
}
```

ตัวอย่างฟังก์ชันที่มีการส่งค่าแต่ไม่คืนค่ากลับ

```
#include <stdio.h>

void get_Fx(int x);

main()
{int first, second, third;

printf("\n F(X) = 3X + 10 if X > 0\n");
printf("\n F(X) = 10 if X = 0\n");
printf("\n Enter 3 values\n");
scanf("%d %d %d", &first, &second, &third);
```

```
get_Fx(first);
get_Fx(second);
get_Fx(third);
}
```

```
void get_Fx(int x)
{ if (x > 0)
    printf("F(%d) is %d\n", x, (3*x) + 10);
else
    printf("F(%d) is 10\n", x);
}
```

- ฟังก์ชันสามารถเรียกใช้งานได้หลายครั้ง โดยส่งอาร์กิวเมนต์ที่แตกต่างกันไปยังฟังก์ชัน
- หากฟังก์ชันไม่มีการส่งคืนค่ากลับ จะระบุ return type เป็น **void**
- เมื่อฟังก์ชันระบุ return type เป็น void ไม่จำเป็นต้องมีคำสั่ง return ในฟังก์ชัน

ตัวอย่างฟังก์ชันไม่ส่งค่าและไม่คืนค่ากลับ

```
#include <stdio.h>
```

```
void printMessage (void)
{
    printf ("Programming is fun.\n");
}
```

```
int main (void)
{
    printMessage ();

    return 0;
}
```

- หากไม่ต้องการประกาศฟังก์ชัน (function prototype) ผู้เขียนโปรแกรมสามารถทำได้โดยการแทนที่ส่วนของการประกาศฟังก์ชันด้วยส่วนของการนิยามฟังก์ชัน

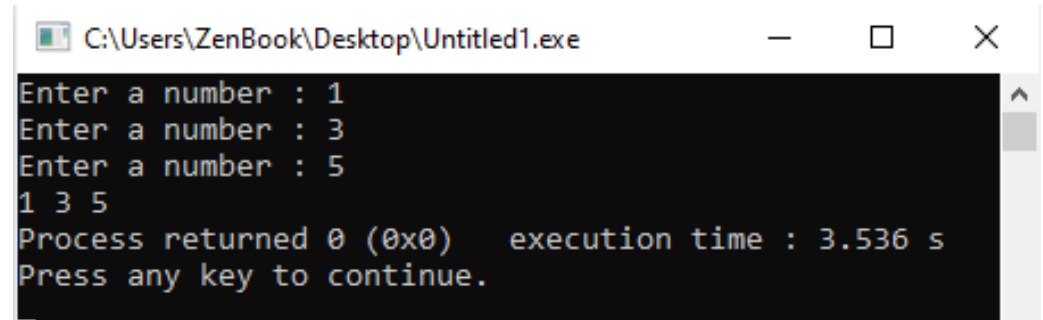
ตัวอย่างฟังก์ชันไม่ส่งค่าแต่ส่งคืนค่ากลับ

```
#include <stdio.h>
```

```
int getvalue();
```

```
int main() {  
    int a, b, c;  
  
    a = getvalue();  
    b = getvalue();  
    c = getvalue();  
  
    printf("%d %d %d", a, b, c);  
    return 0;  
}
```

```
int getvalue()  
{ int x;  
  printf("Enter a number : ");  
  scanf("%d", &x);  
  return x;  
}
```

A screenshot of a Windows command prompt window titled 'C:\Users\ZenBook\Desktop\Untitled1.exe'. The window shows the output of a C program. It prompts the user to 'Enter a number' three times, with inputs 1, 3, and 5. The output then shows '1 3 5' on a single line, followed by 'Process returned 0 (0x0) execution time : 3.536 s' and 'Press any key to continue.'.

```
C:\Users\ZenBook\Desktop\Untitled1.exe  
Enter a number : 1  
Enter a number : 3  
Enter a number : 5  
1 3 5  
Process returned 0 (0x0) execution time : 3.536 s  
Press any key to continue.
```



แบบฝึกหัด

1. จากโปรแกรมต่อไปนี้ จงแบ่งเป็นฟังก์ชันให้ได้มากที่สุด

```
#include<stdio.h>

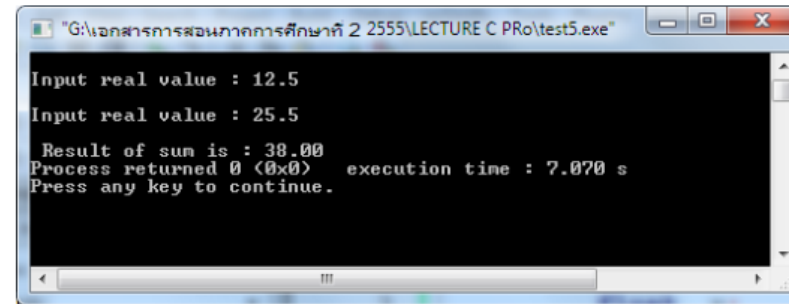
int main()
{ float a1, a2, sumVal;

    printf("Input real value : ");
    scanf("%f", &a1);

    printf("Input real value : ");
    scanf("%f", &a2);

    sumVal = a1 + a2;

    printf(" Result of sum is : %.2f", sumVal);
}
```



2. จงเขียนฟังก์ชันแปลงค่าอุณหภูมิจากองศาเซลเซียสเป็นฟาเรนไฮต์ กำหนดให้ $F = (5/9)*C-32$

การรับและส่งอาร์เรย์ระหว่างฟังก์ชัน

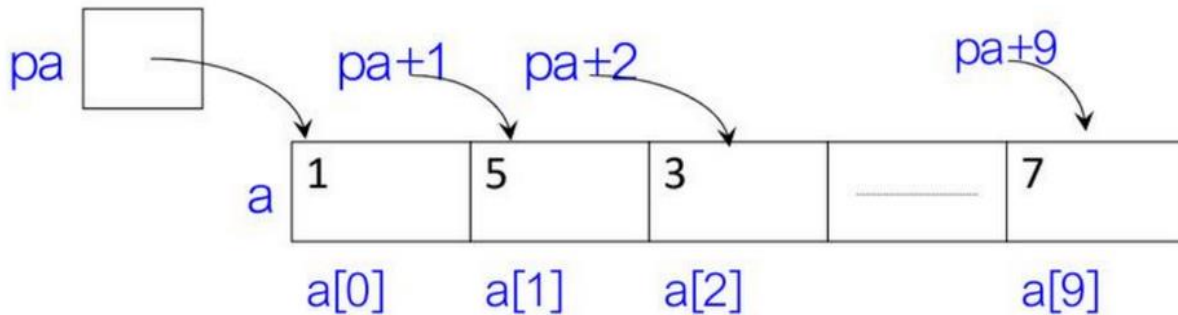
- ฟังก์ชันในภาษาซีอนุญาตให้ผู้เขียนโปรแกรมสามารถรับและส่งอาร์เรย์ (array) ระหว่างฟังก์ชันได้ โดยการระบุพารามิเตอร์ของฟังก์ชันเป็นชนิดข้อมูลแบบอาร์เรย์
- อย่างไรก็ตาม เนื่องจากฟังก์ชันอนุญาตให้ส่งกลับได้เพียงค่าเดียวจึงจะต้องมีวิธีส่งกลับอาร์เรย์ด้วยโอเปอเรเตอร์ * เพื่อให้โปรแกรมภาษาซีส่งกลับเป็นค่าแอดเดรส (address) ของอาร์เรย์แทนที่การส่งกลับด้วยค่าของอาร์เรย์ตรงๆ

int * คือโอเปอเรเตอร์ที่ใช้เข้าถึงแอดเดรสของ
 อาร์เรย์ชนิดจำนวนเต็ม

```
int * func (char [], int [], ..... )  
{ int x[] = {1, 2, 3};  
  .....  
  return x;  
}
```

ความสัมพันธ์ระหว่างโอเปอเรเตอร์ * และอาร์เรย์

- ตัวแปรที่นำหน้าด้วย * เช่น *pb จะมีความสามารถพิเศษในการเก็บค่าแอดเดรสของอาร์เรย์ได้ ดังนั้นการเข้าถึงสมาชิกของอาร์เรย์ เช่นอาร์เรย์ a สามารถทำได้โดยอ้างอิงผ่านตัวแปรดังกล่าวในลักษณะเดียวกับการเข้าถึงสมาชิกในอาร์เรย์ปกติทั่วไป



```
int main() {
    int a[] = {1, 5, 3, 2, 7, 7, 5, 6, 1, 7};
    int *pb, i;

    pb = a;

    for(i=0; i<9; i++)
    { printf("%d ", pb[i]);
    }

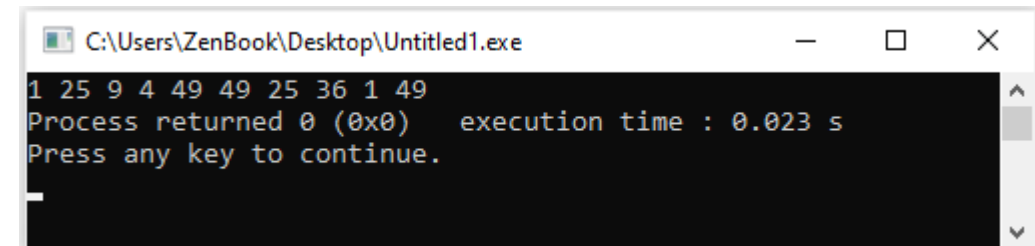
    return 0;
}
```

ตัวอย่างการรับส่งอาร์เรย์ระหว่างฟังก์ชัน

```
#include <stdio.h>
```

```
int * pow_ar(int x[], int n)
{ int i;
  for(i=0; i<n; i++)
  { x[i] = x[i] * x[i];
  }
  return x;
}
```

```
int main() {
  int a[] = {1, 5, 3, 2, 7, 7, 5, 6, 1, 7};
  int *pb, i, n;
  n = sizeof(a) / sizeof(int);
  pb = pow_ar(a, n);
  for(i=0; i<n; i++)
  { printf("%d ", pb[i]);
  }
  return 0;
}
```

A screenshot of a Windows command prompt window titled 'C:\Users\ZenBook\Desktop\Untitled1.exe'. The window shows the output of a program: '1 25 9 4 49 49 25 36 1 49', followed by 'Process returned 0 (0x0) execution time : 0.023 s' and 'Press any key to continue.'.

```
C:\Users\ZenBook\Desktop\Untitled1.exe
1 25 9 4 49 49 25 36 1 49
Process returned 0 (0x0)   execution time : 0.023 s
Press any key to continue.
```



แบบฝึกหัด

จงเขียนฟังก์ชัน `reverse()` เพื่อรับค่าสตริง (string) `s` จากนั้นเรียงกลับทิศ (reverse) สตริงดังกล่าว พร้อมส่งคืนฟังก์ชัน `main ()` เพื่อแสดงผลลัพธ์ออกทางหน้าจอ



ตัวแปรโกลบอลและตัวแปรโลคัล

```
#include <stdio.h>
```

```
void my_func();
```

```
int main()  
{  
    double x = 1.1;  
    printf("In main, x = %.2f \n", x);  
    my_func();  
    printf("In main, x = %.2f \n", x);  
    return 0;  
}
```

```
void my_func()  
{  
    double x;  
    x = 2.5;  
    printf("In my_func, x = %.2f \n", x);  
}
```

x
1.1

x
2.5

- ในภาษาซี ตัวแปรที่ประกาศภายในฟังก์ชันจะถูกเรียกว่าตัวแปรโลคัล (local variable) โดยจะมีขอบเขตมองเห็นของตัวแปรดังกล่าวได้เฉพาะภายในฟังก์ชันที่ประกาศ

- ภาษาซีอนุญาตให้มีการประกาศตัวแปรที่เข้าถึงได้จากทุกฟังก์ชันในโปรแกรม เรียกว่าตัวแปรโกลบอล (global variable) โดยเพียงแค่ประกาศตัวแปรเหนือฟังก์ชัน main

การประกาศตัวแปรโกลบอลในภาษาซี

```
#include <stdio.h>
```

```
double x;           // global variable declare
```

```
void my_func();
```

```
int main()
{ x = 1.1;
  printf("In main, x = %.2f \n", x);
  my_func();
  printf("In main, x = %.2f \n", x);
  return 0;
}
```

```
void my_func()
{ x = 2.5;
  printf("In my_func, x = %.2f \n", x);
}
```

X





คำถาม ?

```
double x;
```

```
void func();
```

```
int main()  
{ x = 1.1;  
  func();
```

```
  printf("x = %.2f\n", x);  
}
```

ผลลัพธ์ของ printf() บรรทัดนี้ คือ

```
void func()  
{ double x;  
  x = 3.0;  
  printf("x = %.2f\n", x);  
}
```

ผลลัพธ์ของ printf() บรรทัดนี้ คือ

สรุปเรื่องฟังก์ชันในภาษาซี

- ฟังก์ชันเป็นความสามารถของภาษาซีที่อนุญาตให้ผู้เขียนโปรแกรมสามารถแบ่งโปรแกรมออกเป็นโปรแกรมย่อย (subprogram) เพื่อลดความซ้ำซ้อน และสามารถนำไป reuse ในโปรแกรมอื่นได้
- การรับส่งค่าระหว่างฟังก์ชันจะต้องมีการประกาศจำนวนและชนิดของข้อมูลให้ชัดเจน ซึ่งขึ้นอยู่กับผู้เขียนโปรแกรมว่าจะออกแบบการรับส่งค่าข้อมูลระหว่างฟังก์ชัน อย่างไรก็ตามการส่งคืนค่ากลับจากฟังก์ชันทำได้เพียง 1 ค่าเท่านั้น
- การส่งคืนค่ากลับของอาร์เรย์และสตริงจากฟังก์ชัน จะต้องใช้โอเปอเรเตอร์ * เพื่อให้สามารถส่งค่ากลับพร้อมกันได้หลายค่า
- โดยทั่วไป ขอบเขตการมองเห็นของตัวแปรจะถูกกำหนดให้อยู่ภายในฟังก์ชันที่ประกาศตัวแปร อย่างไรก็ตาม ภาษาซีอนุญาตให้สามารถประกาศตัวแปรแบบโกลบอลเพื่อให้ทุกฟังก์ชันสามารถเข้าถึงตัวแปรดังกล่าวได้